<Next.js 16 />
<React 19 />
<TypeScript 5 />
<Prisma ORM />
<PostgreSQL />

TECHNICAL DEVELOPER DOCUMENTATION

NEXUS HRMS

Human Resource Management System

Comprehensive technical documentation for the NEXUS HRMS platform, covering system architecture, API specifications, database schema, frontend architecture, security model, deployment procedures, and development guidelines for engineering teams.

Version 2.0

May 2026

v2.0

Table of Contents

1. System Architecture Overview

NEXUS HRMS is built on a modern, full-stack web architecture designed for scalability, maintainability, and developer productivity. The platform follows a layered architecture pattern that cleanly separates concerns across the presentation, API, and data tiers, enabling independent evolution of each layer while maintaining a cohesive system. The architecture is optimized for the unique demands of a Human Resource Management System, including multi-tenant data isolation, role-based access control, and real-time collaboration features.

1.1 Technology Stack

The technology stack was carefully selected to balance developer experience, performance, and long-term maintainability. The frontend is powered by **Next.js 16** with **React 19**, providing a component-driven UI framework with server-side rendering capabilities. **TypeScript 5** is used throughout the codebase, ensuring type safety and reducing runtime errors. The UI layer leverages **Tailwind CSS 4** for utility-first styling and **shadcn/ui** for accessible, customizable component primitives. On the backend, Next.js API Routes handle server-side logic, while **Prisma ORM** provides a type-safe database access layer with automatic migration management. The primary database is **PostgreSQL**, hosted on **Neon serverless** infrastructure for automatic scaling and zero-administration operations. Client-side state management is handled by **Zustand**, a lightweight and predictable state container that integrates seamlessly with React hooks.

Layer	Technology	Purpose
Frontend Framework	Next.js 16 + React 19	SSR, routing, component architecture
Language	TypeScript 5	Type safety across full stack
Styling	Tailwind CSS 4 + shadcn/ui	Utility-first CSS and accessible components
Backend	Next.js API Routes	RESTful endpoints, server logic
ORM	Prisma	Type-safe DB access, migrations
Database	PostgreSQL (Neon)	Relational data, multi-tenant isolation
State Management	Zustand	Client-side global state
Deployment	Vercel + GitHub CI/CD	Edge deployment, automatic previews

Table 1: Technology Stack Overview

1.2 Architecture Layers

The system follows a three-tier layered architecture that enforces strict separation of concerns. The **Presentation Layer** encompasses all React components, pages, and the Zustand store that manages client-side state. Components are organized into modular HRMS domain modules (e.g., recruitment, payroll, attendance) with a shared UI component library. The **API Layer** consists of Next.js API route handlers that process HTTP requests, validate inputs, enforce RBAC policies, and coordinate business logic. This layer also implements demo data fallbacks for development and presentation environments. The **Data Layer** is managed by Prisma ORM, which maps TypeScript models to PostgreSQL tables, handles migrations, and provides connection pooling via the Neon serverless driver. Each layer communicates only with its adjacent layer, preventing tight coupling and enabling independent testing and deployment.

The communication flow follows a strict unidirectional pattern: user interactions in the Presentation Layer trigger API calls to the API Layer, which queries or mutates data through the Data Layer, and responses flow back up through the stack. This pattern ensures that business logic remains centralized in the API layer, database queries are always parameterized through Prisma, and the frontend remains a thin rendering layer. Cross-cutting concerns such as authentication, logging, and error handling are implemented as middleware and utility modules that span the API layer.

2. Project Setup and Configuration

2.1 Prerequisites

Before setting up the NEXUS HRMS development environment, ensure the following prerequisites are installed and properly configured on your system. These tools form the foundation of the build toolchain and runtime environment. Node.js version 20 or higher is required for compatibility with Next.js 16 and its server components architecture. The project uses **bun** as its primary package manager for faster dependency resolution and installation, though npm is also supported as a fallback. A PostgreSQL database instance is required; for local development, a Neon serverless PostgreSQL instance is recommended, but any PostgreSQL 14+ server will work. Git is required for version control and the CI/CD pipeline integration.

Tool	Minimum Version	Purpose
Node.js	20.0+	JavaScript runtime and build toolchain
bun	1.0+	Package manager (preferred over npm)
PostgreSQL	14+	Primary database (or Neon serverless)

Git	2.30+	Version control and CI/CD integration
TypeScript	5.0+	Type-safe development (installed via bun)

Table 2: Development Prerequisites

2.2 Environment Variables

The application relies on environment variables for database connectivity, authentication configuration, and feature flags. All environment variables are defined in a **.env.local** file at the project root. This file is excluded from version control via **.gitignore** to prevent credential leakage. A template file **.env.example** is provided in the repository with all required variable names and placeholder values. When deploying to Vercel, these variables are configured through the Vercel dashboard or the CLI.

Variable	Required	Description
POSTGRES_URL	Yes	Neon serverless PostgreSQL connection string
POSTGRES_PRISMA_URL	Yes	Prisma-accelerated connection string
POSTGRES_URL_NON_POOLING	Yes	Direct connection for migrations
NEXTAUTH_SECRET	Yes	Secret key for NextAuth.js session encryption
NEXTAUTH_URL	Yes	Base URL of the application
NODE_ENV	No	Environment mode (development/production)

Table 3: Environment Variables

2.3 Installation Steps

Setting up the development environment follows a straightforward sequence. First, clone the repository from GitHub and navigate to the project directory. Install dependencies using bun (or npm as an alternative). Copy the environment template and configure your local variables. Run the Prisma migration to initialize the database schema, then seed the database with demo data for development. Finally, start the development server to begin working.

```
git clone https://github.com/org/nexus-hrms.git
cd nexus-hrms
bun install
cp .env.example .env.local
```

```
# Edit .env.local with your database credentials
bunx prisma migrate dev
bunx prisma db seed
bun dev
```

2.4 Development Workflow

The development workflow follows a feature-branch strategy integrated with GitHub Actions CI/CD. Developers create feature branches from the **main** branch, implement changes with accompanying tests, and submit pull requests for code review. The CI pipeline automatically runs linting, type checking, unit tests, and build verification on every push. Merge to main triggers automatic deployment to Vercel. Preview deployments are created for every pull request, enabling reviewers to test changes in an isolated environment before merging. The local development server supports hot module replacement for instant feedback during UI development, and Prisma Studio provides a visual interface for database inspection during development.

2.5 Build and Deployment Process

The production build process is handled by Next.js and Vercel. Running **next build** generates an optimized production bundle with server-side rendered pages, static generation where applicable, and edge-optimized API routes. Vercel automatically performs builds on push to main and deploys to its global edge network. The build output includes chunked JavaScript bundles, CSS files, and serverless function packages for API routes. Environment variables are injected at build time for public variables and at request time for server-side secrets. Database migrations are run as a pre-deploy step through the Vercel build command configuration.

3. API Documentation

The NEXUS HRMS API is built on Next.js API Routes, following RESTful conventions with JSON request and response payloads. All endpoints enforce role-based access control and company-level data isolation. The API implements consistent patterns for pagination, error handling, and demo data fallbacks to ensure a uniform developer experience across all modules. This section documents each API module, its endpoints, and the common patterns that apply across the entire API surface.

3.1 Authentication API (/api/auth)

Authentication is managed through NextAuth.js with JWT-based sessions. The authentication endpoints handle user login, session management, and role resolution. Upon successful authentication, a JWT token is issued containing the user ID, company ID, and assigned role. This token is validated on every

subsequent API request through the authentication middleware. The session includes the user role, which determines module visibility and API access permissions.

Endpoint	Method	Description
/api/auth/signin	POST	Authenticate user with email/password
/api/auth/signout	POST	Invalidate current session
/api/auth/session	GET	Retrieve current session details
/api/auth/callback	GET	OAuth callback handler

Table 4: Authentication Endpoints

3.2 Employee Management API (/api/employees)

The Employee Management API provides full CRUD operations for employee records, which serve as the central entity in the HRMS system. Each employee belongs to a company, is assigned to a department and branch, and has an associated user account for system access. The API supports list operations with pagination, filtering by department, branch, and status, as well as bulk import and export functionality. Employee records maintain an audit trail of all changes for compliance purposes.

Endpoint	Method	Description
/api/employees	GET	List employees with pagination and filters
/api/employees	POST	Create a new employee record
/api/employees/[id]	GET	Retrieve employee details by ID
/api/employees/[id]	PUT	Update employee information
/api/employees/[id]	DELETE	Deactivate an employee record
/api/employees/import	POST	Bulk import employees from CSV/Excel
/api/employees/export	GET	Export employee data to CSV/Excel

Table 5: Employee Management Endpoints

3.3 Recruitment/ATS API (/api/jobs, /api/candidates, /api/interviews)

The Recruitment and Applicant Tracking System (ATS) API manages the complete hiring lifecycle from job requisition to candidate onboarding. The API is divided into three sub-modules: Jobs, Candidates, and Interviews. Jobs represent open positions with requirements, salary ranges, and posting details. Candidates are applicants who have applied to one or more jobs, with their resume data, application status, and evaluation scores. Interviews track the scheduling, feedback, and outcomes of candidate evaluations. The system supports kanban-style pipeline management with configurable stage workflows.

Endpoint	Method	Description
/api/jobs	GET	List open job positions with filters
/api/jobs	POST	Create a new job requisition
/api/jobs/[id]	PUT	Update job posting details
/api/candidates	GET	List candidates with pipeline stage filters
/api/candidates	POST	Add a new candidate to the pipeline
/api/candidates/[id]	PUT	Update candidate status or move pipeline stage
/api/interviews	GET	List scheduled interviews
/api/interviews	POST	Schedule a new interview
/api/interviews/[id]/feedback	POST	Submit interview feedback and rating

Table 6: Recruitment/ATS Endpoints

3.4 Time Management API (/api/attendance, /api/leaves, /api/shifts)

The Time Management API covers three interconnected domains: attendance tracking, leave management, and shift scheduling. Attendance records capture daily clock-in/clock-out events with location data for remote verification. Leave management handles leave requests, approvals, balance tracking, and policy enforcement by leave type. Shift scheduling supports recurring and ad-hoc shift assignments with conflict detection and team-level visibility. All time management endpoints support date-range queries and generate summary reports for payroll integration.

Endpoint	Method	Description
/api/attendance	GET	List attendance records by date range
/api/attendance/clock-in	POST	Record clock-in event with location

/api/attendance/clock-out	POST	Record clock-out event
/api/leaves	GET	List leave requests with status filter
/api/leaves	POST	Submit a new leave request
/api/leaves/[id]/approve	PUT	Approve or reject a leave request
/api/shifts	GET	List shift schedules by team and date
/api/shifts	POST	Create or update shift assignments

Table 7: Time Management Endpoints

3.5 Payroll API (/api/payroll)

The Payroll API manages salary computation, payslip generation, and payroll processing cycles. It integrates with attendance and leave data to calculate accurate monthly compensation including deductions, allowances, and tax withholdings. Payroll runs are processed on a monthly cycle with lock-and-approve workflows to ensure data integrity. The API supports multi-currency payroll, configurable salary components, and statutory compliance reports. Payslips can be generated as PDF documents and distributed to employees via the system.

3.6 Additional Module APIs

NEXUS HRMS includes several additional API modules that cover the full spectrum of HR operations. The **Asset Management API (/api/assets)** handles company asset inventory, assignment to employees, and maintenance tracking. The **Travel and Expense API (/api/travel, /api/expenses)** manages travel requests, expense submissions, and approval workflows with receipt attachments. The **Helpdesk API (/api/tickets)** provides an internal ticketing system for HR service requests with priority routing and SLA tracking. The **Workflow API (/api/workflows)** enables configurable approval and automation workflows that can be attached to any HR process. Finally, the **Analytics API (/api/analytics, /api/dashboard)** aggregates data across all modules to provide real-time dashboards, trend analysis, and exportable reports for HR decision-making.

3.7 Common API Patterns

All API endpoints follow consistent patterns for a uniform developer experience. **Pagination** is implemented using cursor-based pagination with limit and offset parameters. Responses include metadata with total count, current page, and page size. **Error handling** uses standard HTTP status codes with a consistent JSON error envelope containing an error code, human-readable message, and optional details array for validation errors. **Demo data fallbacks** ensure that the UI always has data to display;

when the database is empty (such as in a fresh development environment), API routes return pre-configured demo data that matches the expected response schema. This pattern is controlled by the **DEMO_MODE** environment variable and is automatically activated in development environments.

```
// Standard error response format
{ "error": { "code": "VALIDATION_ERROR", "message": "Invalid request",
"details": [...] } }
```

```
// Standard pagination response format
{ "data": [...], "pagination": { "total": 150, "page": 1, "pageSize": 20 } }
```

4. Database Schema

The NEXUS HRMS database schema is defined using Prisma schema files and deployed through Prisma Migrate. The schema follows a multi-tenant architecture where all data is scoped to a Company entity, ensuring complete data isolation between tenants. The schema is organized into logical domains that map to the application modules. This section provides an overview of the entity relationships and details of the core models across each domain.

4.1 Entity Relationship Overview

The central entity in the schema is the **Company**, which acts as the tenant root. All major entities reference the Company through a foreign key, establishing the multi-tenant boundary. The **User** entity represents a system account and is linked to both a Company and optionally to an **Employee** record. The Employee entity connects to Department, Branch, and various operational entities such as Attendance, Leave, Payroll, and Asset. The Recruitment domain links Jobs to Candidates and Interviews. Workflow definitions and instances are polymorphic, capable of attaching to any entity type through generic reference fields. Audit and notification models track system events across all domains.

4.2 Core Models

Model	Key Fields	Relationships
User	id, email, role, companyId	belongsTo Company, hasOne Employee
Company	id, name, industry, size	hasMany Users, Employees, Departments
Employee	id, firstName, lastName, employeeId	belongsTo Company, Department, Branch
Department	id, name, companyId	belongsTo Company, hasMany Employees

Branch	id, name, address, companyId	belongsTo Company, hasMany Employees
Designation	id, title, level, departmentId	belongsTo Department

Table 8: Core Data Models

4.3 Recruitment Models

Model	Key Fields	Relationships
Job	id, title, type, status, salaryRange	belongsTo Company, hasMany Candidates
Candidate	id, name, email, stage	belongsTo Job, hasMany Interviews
Interview	id, scheduledAt, type, feedback	belongsTo Candidate, hasOne Interviewer

Table 9: Recruitment Models

4.4 Time Models

Model	Key Fields	Relationships
Attendance	id, date, clockIn, clockOut, status	belongsTo Employee
Leave	id, type, startDate, endDate, status	belongsTo Employee, approvedBy User
Shift	id, name, startTime, endTime	belongsTo Company, hasMany Employees
Holiday	id, name, date, companyId	belongsTo Company

Table 10: Time Management Models

4.5 Finance Models

Model	Key Fields	Relationships
Payroll	id, month, year, status, netPay	belongsTo Employee, Company
PayrollComponent	id, type, name, amount	belongsTo Payroll
Expense	id, category, amount, status	belongsTo Employee, approvedBy User
TravelRequest	id, destination, dates, status	belongsTo Employee

Table 11: Finance Models

4.6 Operational and Workflow Models

Operational models cover assets, helpdesk tickets, clients, and vendors. The **Asset** model tracks company property assigned to employees with lifecycle management (allocation, return, disposal). The **Ticket** model implements an internal helpdesk with priority levels, categories, and SLA tracking. **Client** and **Vendor** models manage external entity relationships for procurement and project management contexts. Workflow models provide a generic automation framework. **WorkflowDefinition** defines the steps and conditions for a workflow, while **WorkflowInstance** tracks the runtime state of an active workflow attached to a specific entity. Audit models (**AuditLog**) record all data mutations with before/after snapshots, and **Notification** models track system alerts and their delivery status to users.

5. Frontend Architecture

The NEXUS HRMS frontend is a single-page application built on Next.js 16 with the App Router, providing a rich, interactive user experience. The architecture emphasizes modularity, reusability, and type safety through a well-defined component structure and state management pattern. Every UI module corresponds to an HRMS functional domain, enabling parallel development by multiple teams without conflicts.

5.1 Component Structure

Components are organized into two primary directories. The **src/components/hrms/** directory contains domain-specific components grouped by module, such as recruitment, payroll, attendance, and helpdesk. Each module directory includes page-level components, form components, list/table views, and detail views. The **src/components/ui/** directory houses the shared design system built on shadcn/ui primitives. These include buttons, inputs, modals, tables, and form controls that are styled consistently across the application. The separation ensures that domain components remain focused on business logic while UI components handle presentation concerns. Component props are fully typed with TypeScript interfaces, and complex component states are managed through custom hooks co-located with the component.

Directory	Contents	Pattern
src/components/hrms/dashboard/	Dashboard widgets, KPI cards	Container/Presenter
src/components/hrms/employees/	Employee CRUD forms, list, profile	Module-scoped hooks
src/components/hrms/recruitment/	Job cards, candidate pipeline, interview scheduler	Kanban + forms

src/components/hrms/payroll/	Payslip viewer, run processor	Wizard pattern
src/components/ui/	Button, Input, Table, Modal, Select, etc.	shadcn/ui primitives

Table 12: Component Directory Structure

5.2 Module System

The HRMS modules are orchestrated through a **ModuleKey** system that provides a unified registry of all available modules. Each module is identified by a unique string key (e.g., "employees", "recruitment", "payroll") and is associated with a configuration object containing its display name, icon, route prefix, required role permissions, and the main component to render. The module registry drives the sidebar navigation, route protection, and lazy loading of module components. Modules that the current user does not have permission to access are automatically hidden from navigation and their routes return 403 responses.

5.3 State Management

Client-side state management uses **Zustand** with a modular store architecture. The global store is divided into slices, each managing a specific domain such as authentication state, UI preferences (sidebar open/closed, theme), and the currently active module. Zustand was chosen over Redux for its minimal boilerplate, TypeScript-first API, and excellent React integration through hooks. Server state (API data) is managed through a custom API integration layer rather than cached in Zustand, keeping the client store focused on purely local UI state. This separation prevents stale data issues and simplifies cache invalidation.

5.4 API Integration Layer

All API communication is centralized through **src/lib/api.ts**, which provides typed wrapper functions for every API endpoint. This layer handles authentication token injection, request/response type mapping, error normalization, and retry logic. Each module has a corresponding API namespace (e.g., **api.employees.getList()**, **api.recruitment.createJob()**) that returns typed Promises. The integration layer also implements the demo data fallback mechanism: when an API call fails with a network error and the application is in development mode, the layer returns pre-configured demo data that matches the expected response type. This ensures the UI always has content to display, even during initial development or when the backend is temporarily unavailable.

5.5 Routing, Error Handling, and Responsive Design

Routing follows the Next.js App Router convention with nested layouts. The root layout provides the sidebar navigation and authentication gate, while each module has its own route segment with a module-specific layout. Error handling is implemented through **ModuleErrorBoundary**, a React error boundary component that catches rendering errors within a module and displays a user-friendly fallback UI with a retry button, preventing a single module failure from crashing the entire application. The UI is fully responsive, adapting from desktop (sidebar + multi-column layouts) to tablet (collapsible sidebar) to mobile (bottom navigation, stacked layouts) using Tailwind CSS responsive breakpoints and container queries.

6. Security and Authentication

Security is a foundational concern in the NEXUS HRMS architecture, given the sensitive nature of HR data including personal information, salary details, and performance evaluations. The platform implements defense-in-depth security through role-based access control, company-level data isolation, encrypted sessions, and comprehensive audit logging.

6.1 Role-Based Access Control

The system defines 14 distinct user roles, each with specific module visibility and action permissions. Roles range from super administrators with full system access to employees with self-service-only permissions. The role hierarchy ensures that each user sees only the modules and data relevant to their responsibilities. Role assignments are stored in the User model and resolved at authentication time, then cached in the session JWT token for efficient access control decisions on subsequent requests.

Role	Access Level	Key Permissions
Super Admin	Full system	All modules, all companies, user management
Admin	Company-wide	All modules within assigned company
HR Manager	Department-wide	Employee CRUD, recruitment, payroll view
HR Executive	Limited scope	Attendance, leave management, employee view
Recruiter	Recruitment only	Jobs, candidates, interviews
Payroll Manager	Payroll only	Payroll processing, payslip generation
Manager	Team-level	Team attendance, leave approval, performance
Employee	Self-service	Own profile, attendance, leave requests

Finance	Finance module	Expenses, travel, payroll reports

Table 13: User Roles and Permissions

6.2 Module Visibility Per Role

Each module has a visibility configuration that maps roles to access levels (none, read, write, admin). This configuration is enforced at three levels: UI rendering (modules hidden from navigation), API routing (unauthorized requests return 403), and database queries (queries are scoped to permitted data ranges). For example, an Employee role can view their own attendance and leave records but cannot access other employees records. A Manager role can view and approve leave requests for their direct reports but cannot modify payroll data. This layered enforcement ensures that even if one layer is bypassed, the other layers continue to protect the data.

6.3 Session Management and Data Isolation

Sessions are managed using NextAuth.js with JWT tokens stored in secure, HTTP-only cookies. Tokens have a configurable expiration period (default 24 hours) and are refreshed automatically through silent re-authentication. The JWT payload includes the user ID, company ID, and role, which are validated on every API request. Company-level data isolation is enforced at the Prisma query level through a global middleware that automatically injects **companyId** filters into every database query based on the authenticated session. This ensures that a user in Company A can never access data belonging to Company B, even if they somehow construct a direct API request with a foreign entity ID. All API routes validate that the requested entity belongs to the authenticated user company before returning data.

7. Deployment Guide

NEXUS HRMS is deployed on Vercel, leveraging its global edge network for low-latency content delivery and serverless function execution. The deployment pipeline is fully automated through GitHub integration, with preview environments for every pull request and production deployments triggered by merges to main.

7.1 Vercel Deployment Configuration

The Vercel project is configured through a **vercel.json** file in the repository root and the Vercel dashboard settings. Key configuration includes the framework preset (Next.js), build command (next build), output directory (.next), and environment variable mappings. The project uses the Vercel Neon integration for automatic database provisioning, which creates a new Neon branch for each preview deployment. This ensures that preview environments have isolated databases and never affect

production data. Serverless function configuration includes memory allocation (1024MB default), execution timeout (10 seconds for API routes, 60 seconds for long-running operations), and regional deployment preferences.

7.2 Environment Setup

Production environment variables are configured through the Vercel dashboard under Project Settings. All secret values (database URLs, auth secrets) should be marked as encrypted. The recommended setup includes separate Vercel projects for staging and production, each with its own Neon database instance. Environment variable promotion from staging to production follows a manual approval step to prevent accidental configuration drift. The deployment team should verify that all required environment variables are set before the first production deployment by comparing the Vercel configuration against the .env.example template in the repository.

7.3 Database Provisioning

The database is provisioned through Neon serverless PostgreSQL, which provides automatic scaling, point-in-time recovery, and branching capabilities. For production, a dedicated Neon project is created with a primary compute endpoint in the region closest to the target user base. Database migrations are executed as part of the Vercel build process using the Prisma Migrate deploy command, which applies pending migrations in order and fails the build if any migration encounters an error. The Neon branching feature is used to create preview databases for each pull request, enabling isolated testing of schema changes before they reach production. Connection pooling is handled automatically by the Neon serverless driver, which is configured through the POSTGRES_PRISMA_URL connection string with PgBouncer-compatible parameters.

7.4 Build Optimization and Monitoring

Build optimization is achieved through Next.js automatic code splitting, tree shaking, and image optimization. The build configuration enables SWC minification, removes dead code, and generates source maps for production debugging. Bundle analysis is available through the next-bundle-analyzer plugin, which is run in CI to detect size regressions. Monitoring is configured through Vercel Analytics, which provides real-time performance metrics including Core Web Vitals, serverless function execution times, and error rates. Alerts are configured for function timeout errors, elevated 5xx rates, and database connection pool exhaustion. Log aggregation is handled by Vercel Logs with configurable retention periods and export to external observability platforms.

8. Development Guidelines

8.1 Code Style and Conventions

The codebase follows strict coding conventions enforced through automated tooling. **ESLint** is configured with the Next.js and TypeScript plugin presets, supplemented by custom rules for HRMS-specific patterns such as enforcing company-scoped queries and prohibiting direct Prisma client usage outside of designated repository modules. **Prettier** handles code formatting with a project-wide configuration that ensures consistent indentation (2 spaces), line length (100 characters), and trailing commas. Pre-commit hooks run lint-staged to automatically format and lint modified files before each commit, preventing style inconsistencies from entering the codebase.

8.2 TypeScript Usage

TypeScript is used in strict mode with all compiler checks enabled. The codebase enforces a no-implicit-any policy, requiring explicit type annotations for all function parameters and return types. Complex types are defined in a central **src/types/** directory organized by domain, with shared utility types in a common module. API request and response types are auto-generated from the Prisma schema where possible, and manually defined for custom endpoints. Generic types and utility types (Pick, Omit, Partial) are used extensively to reduce type duplication while maintaining precision. The tsconfig.json extends the Next.js strict configuration with additional path aliases for clean imports.

8.3 Component Development Patterns

React components follow a consistent pattern: typed props interfaces defined immediately above the component, custom hooks for complex state logic, and a clear separation between container (data-fetching) and presenter (display-only) components. Each HRMS module component receives its data through props from a parent container that handles API calls and error states. Form components use controlled inputs with react-hook-form for validation and zod schemas for type-safe form validation. Modal dialogs follow a compound component pattern with Modal.Root, Modal.Trigger, and Modal.Content sub-components for flexible composition.

8.4 API Route Development Patterns

API routes follow a standard structure: authentication check, input validation, business logic, and response formatting. Each route handler is a named export (GET, POST, PUT, DELETE) in a route.ts file within the appropriate API directory. Authentication is enforced through a wrapper function (withAuth) that validates the session and injects the user context. Input validation uses zod schemas that are shared between the API route and the frontend form, ensuring client and server validation rules stay synchronized. Error handling follows a centralized pattern where unexpected errors are caught by a

top-level error boundary and returned as standardized 500 responses with correlation IDs for log tracing.

8.5 Testing Approach

The testing strategy follows the testing pyramid: a broad base of unit tests, a middle layer of integration tests, and a small number of end-to-end tests. **Unit tests** use Vitest for fast execution and cover utility functions, custom hooks, and individual component behavior with mocked API calls. **Integration tests** use the Vitest + Testing Library combination to test component interactions, form submissions, and data flow through the API integration layer. **End-to-end tests** use Playwright to verify critical user journeys such as employee onboarding, leave approval workflows, and payroll processing. API route tests use a test database with seed data, running against a real Prisma client to validate query correctness.

8.6 Git Workflow

The team follows a trunk-based development workflow with short-lived feature branches. The **main** branch is always deployable, and feature branches are created from main with a naming convention of **feature/JIRA-123-brief-description**. Commits should be small, focused, and accompanied by clear messages following the Conventional Commits specification (feat:, fix:, chore:, docs:, refactor:). Pull requests require at least one approval from a code owner, passing CI checks, and no merge conflicts. Squash merging is used to maintain a clean commit history on main. Hotfixes follow the same PR process but are fast-tracked through review with a single required approval. Release tags are created automatically by the CI pipeline on merge to main using semantic versioning.